Parallel ICP Registration

Christian Faccio

July 14, 2026

1. Introduction

ICP Registration

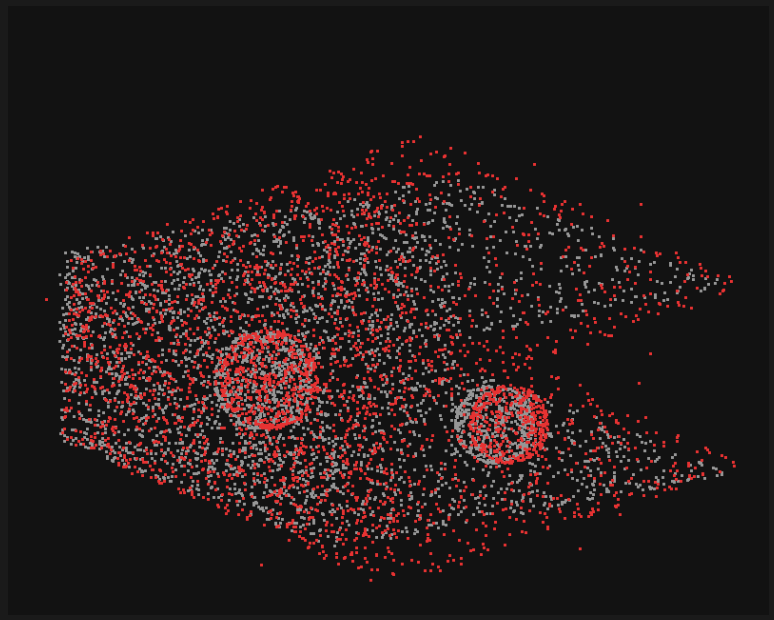
Definition (Registration)

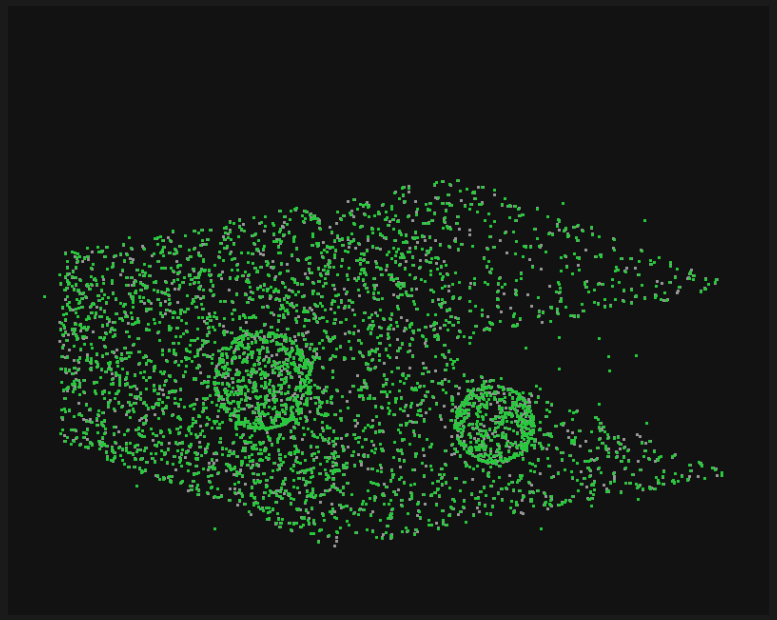
Registration is the process of transforming different sets of data into one coordinate system.

Definition (ICP)

The Iterative Closest Point (ICP) algorithm is a method used to minimize the difference between two clouds of points.

The objective: find the best transformation (rotation and translation) that aligns two point clouds.





The ICP Algorithm (brute force)

Require: source P, target Q, max distance $d_{\max}$, `max_iters`, tolerance τ
Ensure: rigid transform (R, t) aligning P to Q

$X \leftarrow \text{copy}(P)$; $R \leftarrow I$; $t \leftarrow 0$; $e_{\text{prev}} \leftarrow \infty$
for $it = 1$ **to** `max_iters` **do**
  for each point $x_i \in X$ **do** ▷ nearest neighbour + rejection
    $(j, d^2) \leftarrow \textsc{BruteForceNearest}(Q, x_i)$ ▷ scan *all* of Q
    $\text{match}[i] \leftarrow j$ **if** $d^2 \le d_{\max}^2$ **else** $\varnothing$
  end for
  $c_X, c_Q \leftarrow$ centroids of matched pairs; $m \leftarrow \#\text{matches}$
  if $m < 3$ **then break** ▷ under-constrained
  end if
  $H \leftarrow \sum_i (x_i - c_X)(q_j - c_Q)^\top$ ▷ cross-covariance
  $(R_s, t_s) \leftarrow \textsc{Kabsch}(H, c_X, c_Q)$ ▷ SVD solve
  $X \leftarrow R_s X + t_s$ ▷ apply step
  $R \leftarrow R_s R$; $t \leftarrow R_s t + t_s$ ▷ accumulate
  $e \leftarrow \sqrt{\text{SSE}/m}$
  if $|e_{\text{prev}} - e| < \tau$ **then break**
  end if
  $e_{\text{prev}} \leftarrow e$
end for
return (R, t)

Cost: $\textsc{BruteForceNearest}$ scans all N_Q target points $\Rightarrow \mathcal{O}(N_P \cdot N_Q)$ per iteration — the bottleneck the KD-tree removes.

As you can imagine, the time it takes on a single CPU without any optimization or vectorization is soooooooo long (we talk about tens minutes for point clouds of 500000 points).

The first solution is to change data structure to enable a more efficient search: the **KD-Tree**.

2. Serial with KD-Tree

KD-tree data structure

```
1  typedef struct {
2      int point;   /* index into the source PointCloud */
3      int axis;    /* split axis: 0=x, 1=y, 2=z */
4      int left;    /* child node indices, or -1 */
5      int right;
6  } KDNode;
7
8  typedef struct {
9      const PointCloud *pts;   /* not owned */
10     int    *perm;            /* working index permutation (owned) */
11     KDNode *nodes;           /* node pool (owned) */
12     int     n;
13     int     root;            /* root node index, or -1 if empty */
14 } KDTree;
```

Baseline

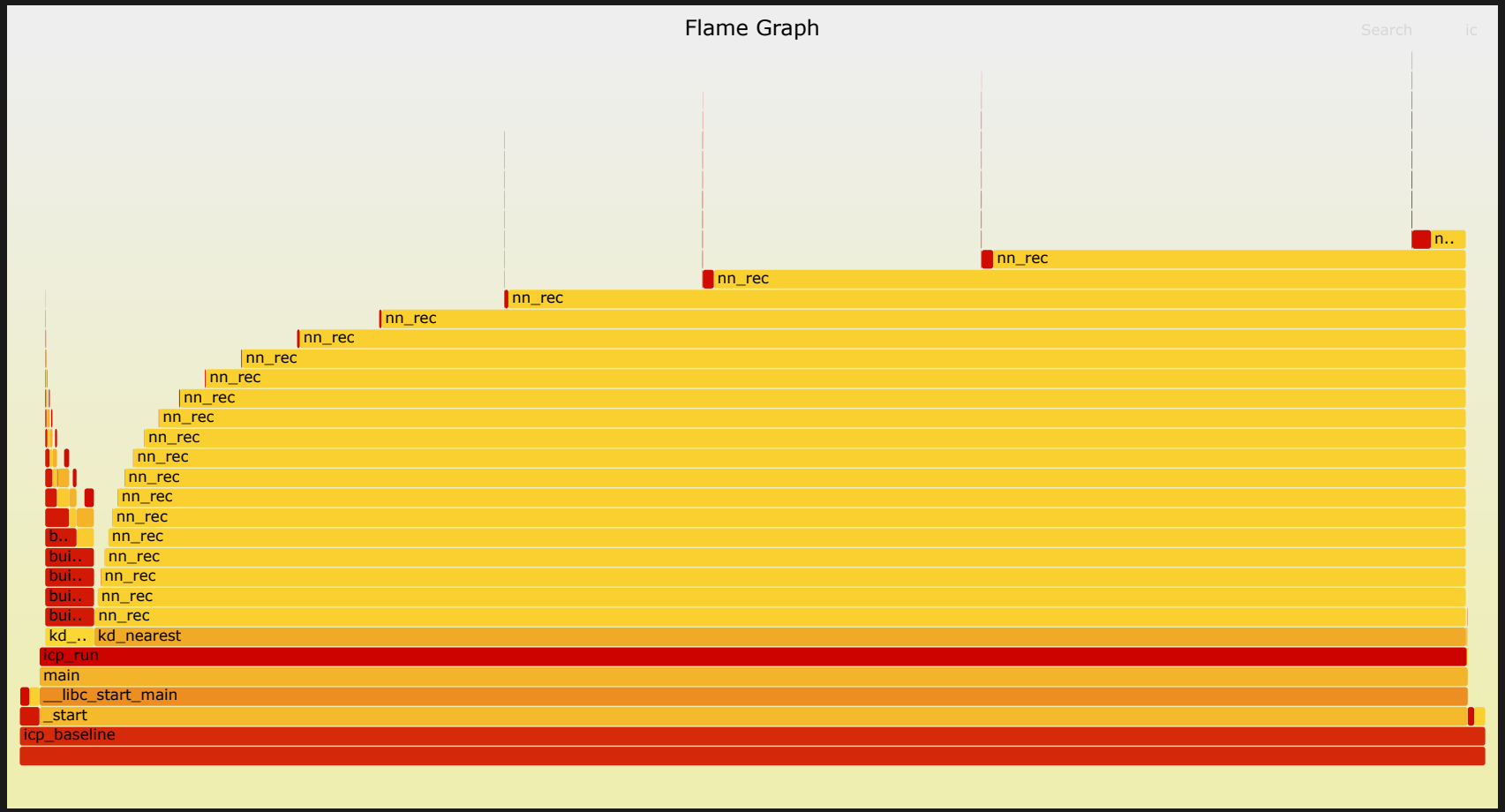
```
cc -std=c11 -Wall -Wextra -O0
```

Output

```
==== Serial ICP (point-to-point, Kabsch/SVD) ====
backend          : KD-tree
target points    : 500000
source points    : 446322  (keep 85%, +5% outliers, noise 0.01 m)
seed             : 12345

ground-truth T   : [-0.4224  0.3500 -0.2812]
recovered    T   : [-0.4227  0.3470 -0.2781]

iterations       : 34
final RMSE       : 0.103125 m  over 440021 pairs
rotation error   : 0.0916 deg
translation err  : 0.0044 m

---- timing (s) ----
scene gen        :   0.0479
icp total        : 308.1603
  setup build    :  10.2658  (3.3%)
  NN search      : 297.1053  (96.4%)
  reduce+solve   :   0.6290  (0.2%)
  transform      :   0.1594  (0.1%)

result           : PASS (converged to ground truth)
```

Perf

```
Performance counter stats for 'CPU(s) 9':

    1043956340356      cycles
    1257070327578      instructions              #    1,20  insn per cycle
      25471311508      cache-references
       2006711140      cache-misses              #    7,878 % of all cache refs
       2718197191      branch-misses
     630087378373      L1-dcache-loads
      33329671250      L1-dcache-load-misses     #    5,29% of all L1-dcache accesses
      19553254733      LLC-loads
        889019703      LLC-load-misses           #    4,55% of all LL-cache accesses

    308,314658233 seconds time elapsed
```


Flame Graph
Search
ic
n..
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
b..
nn_rec
bui..
nn_rec
bui..
nn_rec
bui..
nn_rec
bui..
nn_rec
kd_..
kd_nearest
icp_run
main
__libc_start_main
_start
icp_baseline

With Optimization

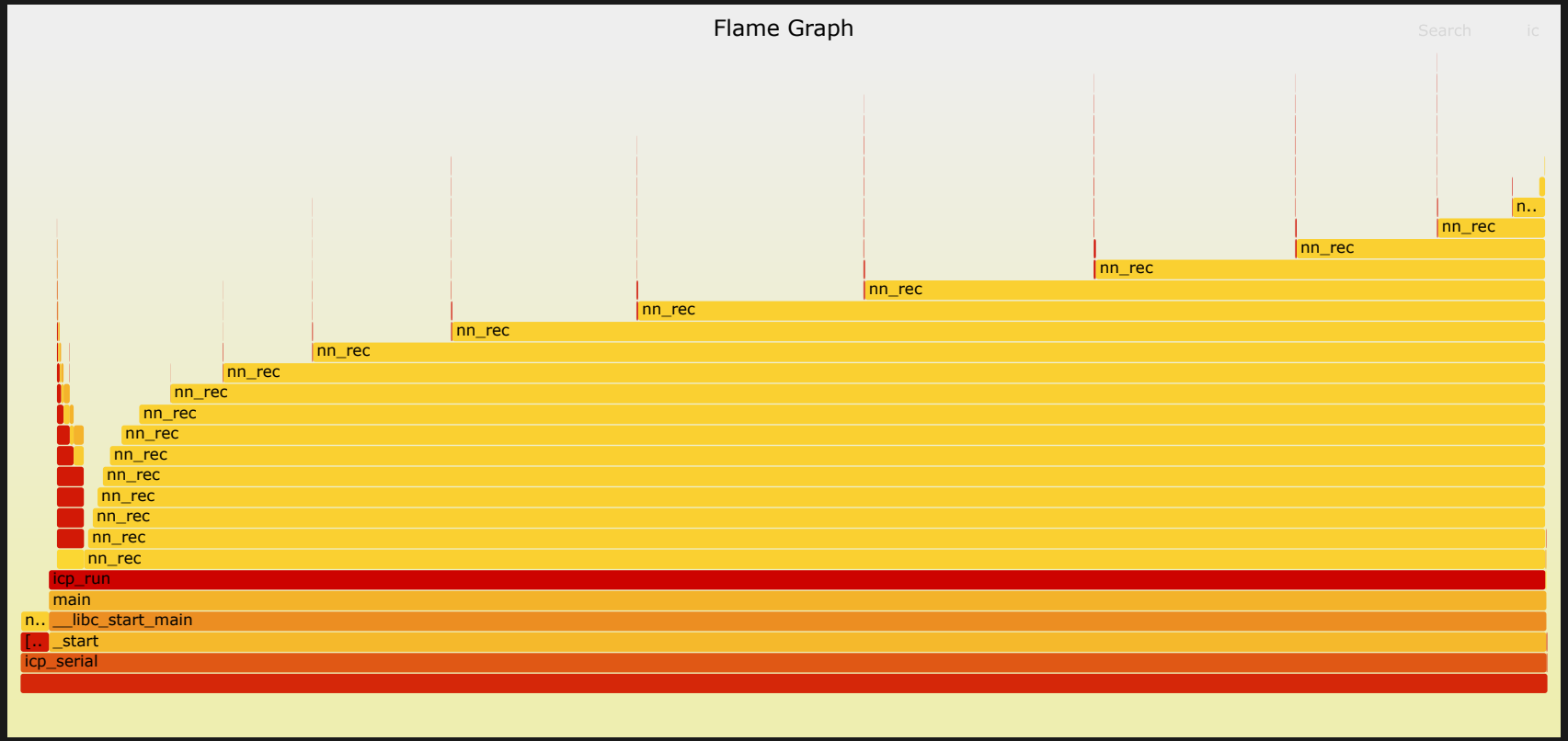
```
cc -std=c11 -Wall -Wextra -O3 -march=native
```

Output

```
==== Serial ICP (point-to-point, Kabsch/SVD) ====
backend          : KD-tree
target points    : 500000
source points    : 446322  (keep 85%, +5% outliers, noise 0.01 m)
seed             : 12345

ground-truth T   : [-0.4224  0.3500 -0.2812]
recovered    T   : [-0.4227  0.3470 -0.2781]

iterations       : 34
final RMSE       : 0.103125 m  over 440021 pairs
rotation error   : 0.0916 deg
translation err  : 0.0044 m

---- timing (s) ----
scene gen        :   0.0241
icp total        : 152.1925
  setup build    :   2.3360  (1.5%)
  NN search      : 149.5116  (98.2%)
  reduce+solve   :   0.3250  (0.2%)
  transform      :   0.0190  (0.0%)

result           : PASS (converged to ground truth)
```

Perf

```
Performance counter stats for 'CPU(s) 2':

     517369746185      cycles
     615443735820      instructions              #    1,19  insn per cycle
      24749045046      cache-references
       1228365214      cache-misses              #    4,963 % of all cache refs
       3154191417      branch-misses
     199029317746      L1-dcache-loads
      31859062460      L1-dcache-load-misses     #   16,01% of all L1-dcache accesses
      20397565788      LLC-loads
        417307915      LLC-load-misses           #    2,05% of all LL-cache accesses

    152,288273603 seconds time elapsed
```


Flame Graph
Search
ic
n..
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
icp_run
main
n..
__libc_start_main
[..
_start
icp_serial

Comments

- **Brute-force:** tens of minutes of computation for 500k points, $\mathcal{O}(N_P \cdot N_Q)$
- **KD-tree non optimized:** 5 minutes, $\mathcal{O}(N_P \log N_Q)$
- **KD-tree optimized:** 2.5 minutes, $\mathcal{O}(N_P \log N_Q)$

Changing the structure for faster search is a big improvement, but nowadays **parallelization** is the key.

3. Vectorization

What does it mean to vectorize?

Nowadays, SIMD capable registers are available on most modern CPUs, which allow to perform the same operation on multiple data points at once. On the Leonardo Booster partition, AVX2 capable registers are available.

There are two possible vectorization strategies:

- **Point level** - Searching the tree is still sequential, but leaves contain multiple points that can be processed in parallel;
- **Query level** - Multiple queries are processed in parallel, each query is still sequentially searching the tree.

v0

The first version uses **point level** vectorization, where the leaves of the KD-tree contain 16 float points (AVX2) that are processed in parallel.

- SoA for the leaves;
- Use of vector types;
- Alignment not possible cause a node is 276 bytes.

KD-tree-V data structure

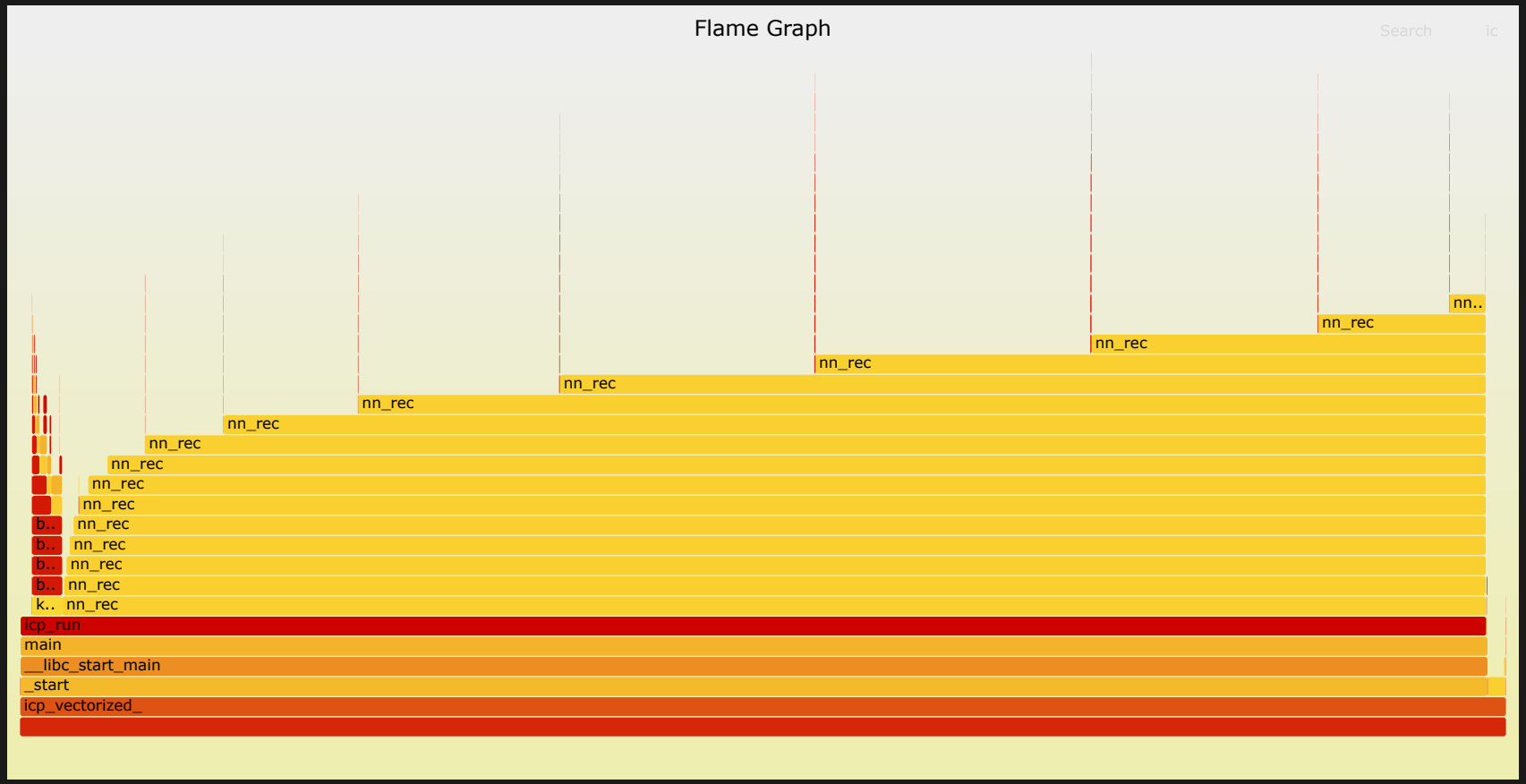
```
typedef struct {
    /* leaf */
    float xs[KD_W], ys[KD_W], zs[KD_W]; /* arrays of points' coordinates */
    int idx[KD_W];                      /* lane -> target point index */
    int count;                          /* -1: internal */

    /* internal */
    int axis;                           /* split axis: 0=x, 1=y, 2=z */
    float split;                        /* routing threshold */
    int left;                           /* child node indices, or -1 */
    int right;
} KDNodeV;

typedef struct {
    const PointCloud *pts;
    int    *perm;                       /* working index permutation */
    KDNodeV *nodes;                     /* node pool */
    int     n;
    int     n_nodes;                    /* number of valid entries in `nodes` */
    int     root;                       /* root node index, or -1 if empty */
} KDTreeV;
```

Output

```
==== Serial ICP (point-to-point, Kabsch/SVD) ====
backend          : KD-tree
target points    : 500000
source points    : 446322  (keep 85%, +5% outliers, noise 0.01 m)
seed             : 12345

ground-truth T   : [-0.4224  0.3500 -0.2812]
recovered    T   : [-0.4227  0.3470 -0.2781]

iterations       : 34
final RMSE       : 0.103125 m  over 440021 pairs
rotation error   : 0.0916 deg
translation err  : 0.0044 m

---- timing (s) ----
scene gen        :   0.0247
icp total        : 110.2029
  setup build    :   2.2572  (2.0%)
  NN search      : 107.1828  (97.3%)
  reduce+solve   :   0.7310  (0.7%)
  transform      :   0.0305  (0.0%)

result           : PASS (converged to ground truth)
```

Perf

```
Performance counter stats for 'CPU(s) 1':

     363961796796      cycles
     149120067352      instructions              #    0,41  insn per cycle
       7727779952      cache-references
       1943536524      cache-misses              #   25,150 % of all cache refs
       3026037796      branch-misses
      48683184247      L1-dcache-loads
       8767870198      L1-dcache-load-misses     #   18,01% of all L1-dcache accesses
       2904251254      LLC-loads
        823335432      LLC-load-misses           #   28,35% of all LL-cache accesses

    110,309730946 seconds time elapsed
```


Flame Graph
Search
ic
nn..
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
nn_rec
b..
nn_rec
b..
nn_rec
b..
nn_rec
b..
nn_rec
k..
nn_rec
icp_run
main
__libc_start_main
_start
icp_vectorized_

v1

The second version uses **query level** vectorization, where multiple queries are processed in parallel, each query is still sequentially searching the tree.

- BFS traversal of the tree
- Usage of a stack instead of recursion
- Masking instead of branching

Output

```
==== Serial ICP (point-to-point, Kabsch/SVD) ====
backend          : KD-tree
target points    : 100000
source points    : 89199  (keep 85%, +5% outliers, noise 0.01 m)
seed             : 12345

ground-truth T   : [-0.4224  0.3500 -0.2812]
recovered    T   : [-0.4302  0.3385 -0.2793]

iterations       : 34
final RMSE       : 0.157003 m  over 86141 pairs
rotation error   : 0.1228 deg
translation err  : 0.0140 m

---- timing (s) ----
scene gen        :   0.0050
icp total        : 127.2728
  setup build    :   0.0853  (0.1%)
  NN search      : 127.1280  (99.9%)
  reduce+solve   :   0.0549  (0.0%)
  transform      :   0.0043  (0.0%)

result           : PASS (converged to ground truth)
```

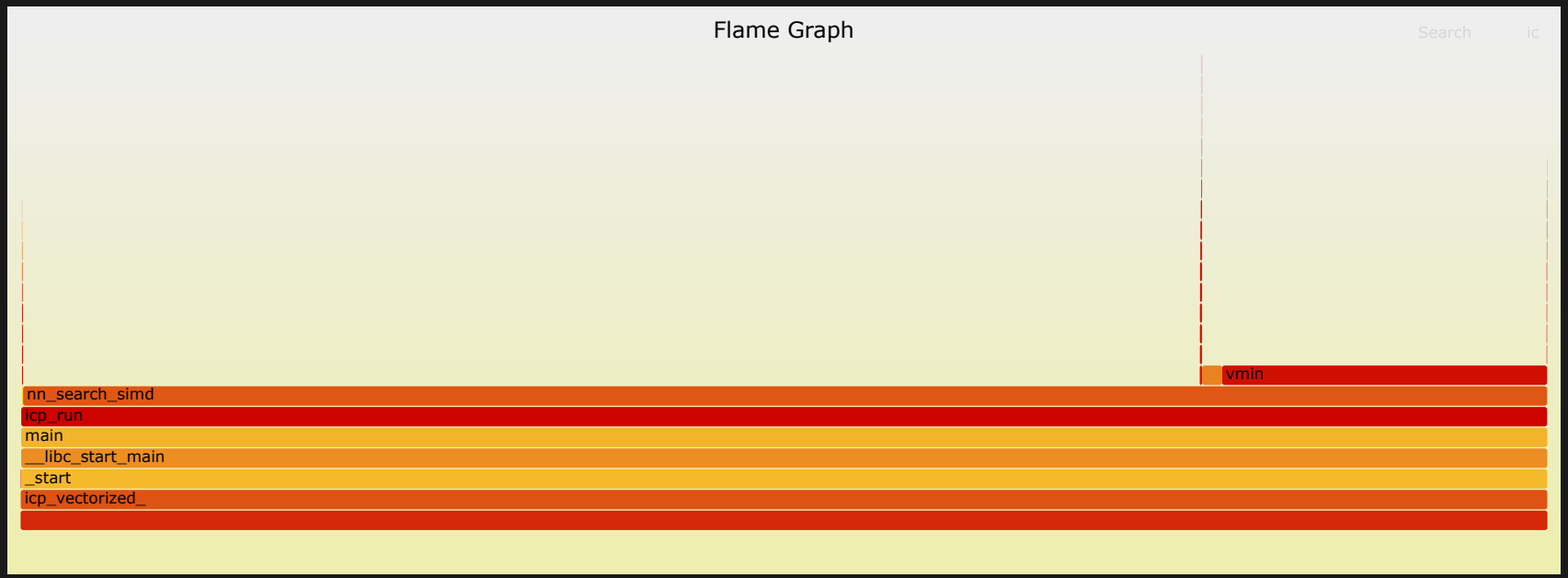
Wait, what happened?

The problem with this approach is that queries are not spatially ordered, so even if multiple queries are processed in parallel, they want to access different parts of the tree, which leads to a lot of **cache misses** and poor performance.

Let's have a look...

Perf

```
Performance counter stats for 'CPU(s) 0':

   420040730809      cycles
   979458976678      instructions              #    2,33  insn per cycle
    10837346914      cache-references
       53828239      cache-misses              #    0,497 % of all cache refs
      366709887      branch-misses
    69930189143      L1-dcache-loads
     8131789786      L1-dcache-load-misses     #   11,63% of all L1-dcache accesses
      636433849      LLC-loads
        9407028      LLC-load-misses           #    1,48% of all LL-cache accesses

   127,294517747 seconds time elapsed
```


Flame Graph
Search
ic
vmin
nn_search_simd
icp_run
main
__libc_start_main
_start
icp_vectorized_

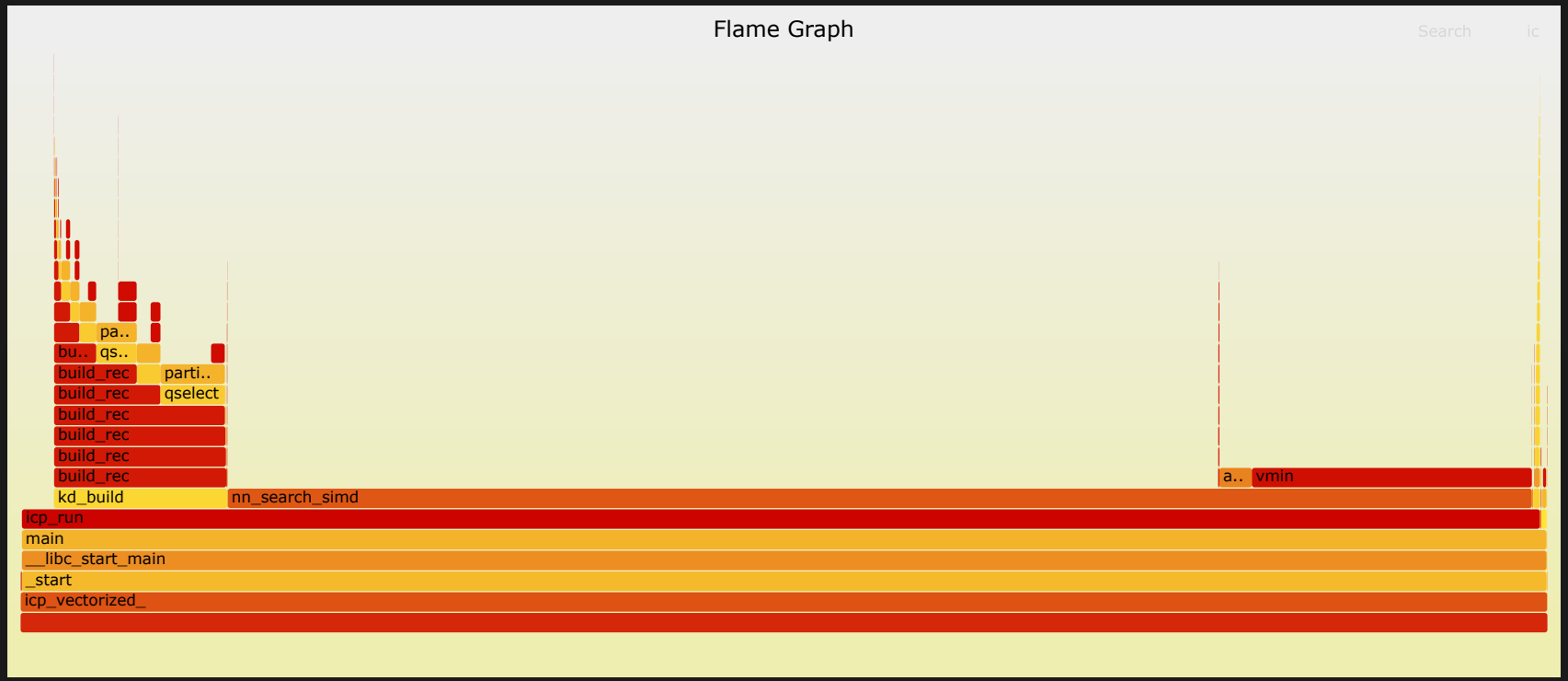
To solve that problem, a spatial ordering of the queries is needed, so that tree divergence is minimized. This ordering is ensured by using a **Morton code** (Z-order curve) to sort the queries before processing them in parallel.

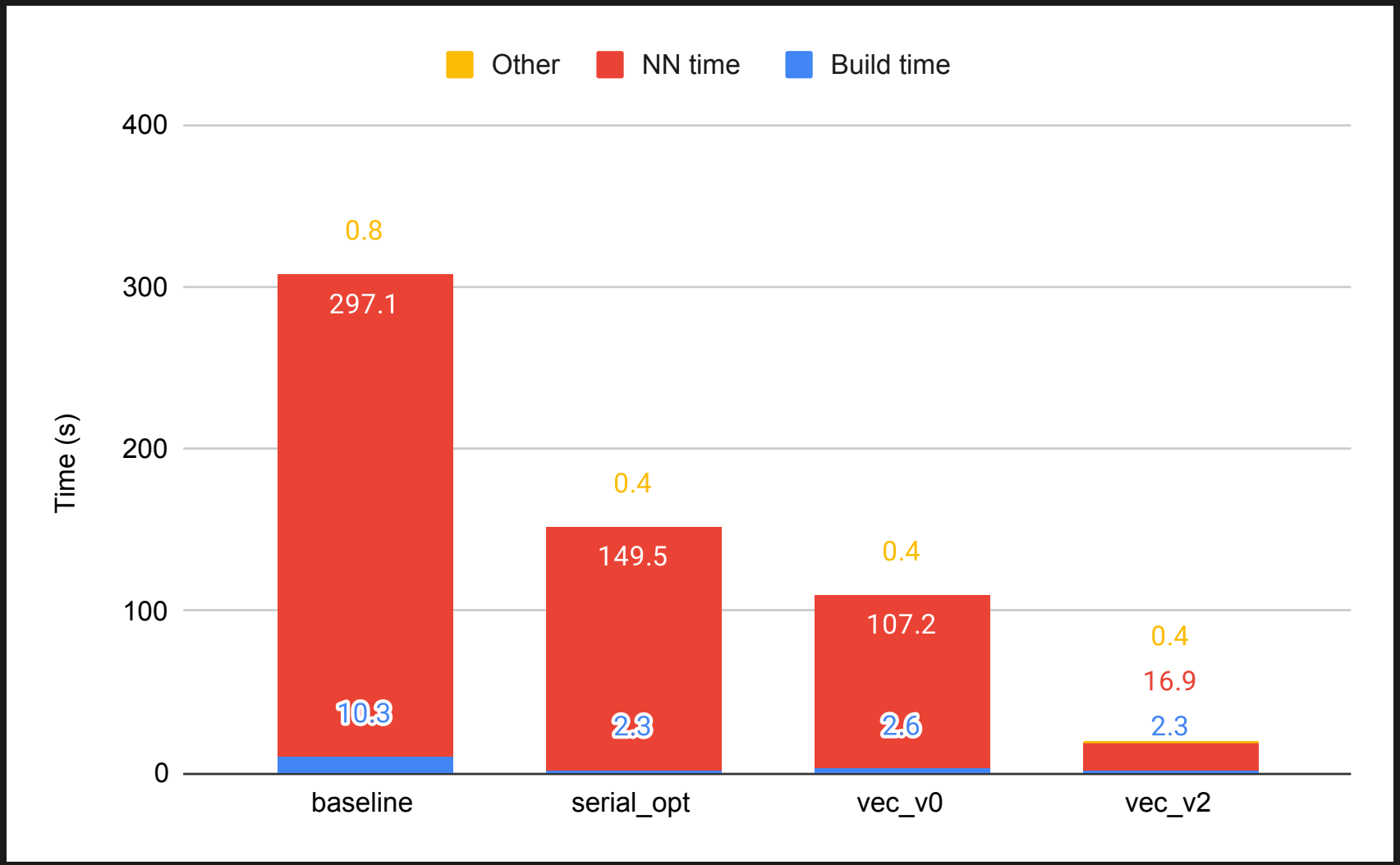
Output

```
==== Serial ICP (point-to-point, Kabsch/SVD) ====
backend          : KD-tree
target points    : 500000
source points    : 446322  (keep 85%, +5% outliers, noise 0.01 m)
seed             : 12345

ground-truth T   : [-0.4224  0.3500 -0.2812]
recovered    T   : [-0.4228  0.3470 -0.2784]

iterations       : 35
final RMSE       : 0.103448 m  over 440003 pairs
rotation error   : 0.0923 deg
translation err  : 0.0041 m

---- timing (s) ----
scene gen        :   0.0243
icp total        :  19.5953
  setup build    :   2.2775  (11.6%)
  NN search      :  16.9184  (86.3%)
  reduce+solve   :   0.3803  (1.9%)
  transform      :   0.0179  (0.1%)

result           : PASS (converged to ground truth)
```

Perf

```
Performance counter stats for 'CPU(s) 16':

      65227592054      cycles
     144161858681      instructions              #    2,21  insn per cycle
        751681982      cache-references
         47679247      cache-misses              #    6,343 % of all cache refs
        121672102      branch-misses
      13792967130      L1-dcache-loads
       3136613071      L1-dcache-load-misses     #   22,74% of all L1-dcache accesses
        313041783      LLC-loads
         12058846      LLC-load-misses           #    3,85% of all LL-cache accesses

     19,691134302 seconds time elapsed
```


Flame Graph
Search
ic
pa..
bu..
qs..
build_rec
parti..
build_rec
qselect
build_rec
build_rec
build_rec
build_rec
kd_build
nn_search_simd
a..
vmin
icp_run
main
__libc_start_main
_start
icp_vectorized_


Other
NN time
Build time
400
300
200
100
0
Time (s)
0.8
297.1
10.3
0.4
149.5
2.3
0.4
107.2
2.6
0.4
16.9
2.3
baseline
serial_opt
vec_v0
vec_v2

4. OMP

OpenMP

The next intuitive step it to parallelize on query level using **OpenMP**, which spawns multiple threads to process the queries in parallel, each traversing the tree sequentially and using SIMD vectorization on the leaves.

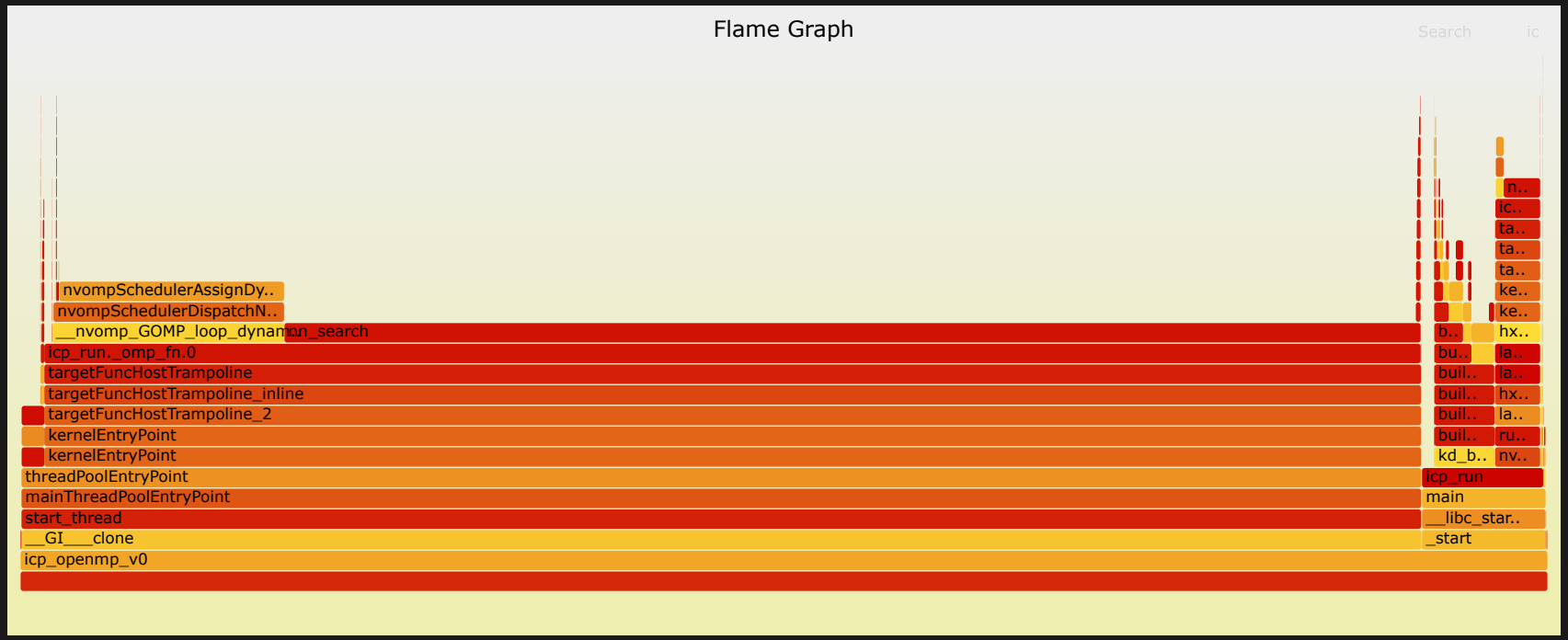
- `schedule(dynamic)`
- `OMP_PROC_BIND=close`
- `OMP_PLACES=cores`

Output

```
==== Serial ICP (point-to-point, Kabsch/SVD) ====
backend          : KD-tree
target points    : 500000
source points    : 446322  (keep 85%, +5% outliers, noise 0.01 m)
seed             : 12345

ground-truth T   : [-0.4224  0.3500 -0.2812]
recovered    T   : [-0.4227  0.3470 -0.2781]

iterations       : 34
final RMSE       : 0.103125 m  over 440021 pairs
rotation error   : 0.0916 deg
translation err  : 0.0044 m

---- timing (s) ----
scene gen        :   0.0239
icp total        :   4.4144
  setup build    :   2.2027  (49.9%)
  NN search      :   1.7548  (39.8%)
  reduce+solve   :   0.4300  (9.7%)
  transform      :   0.0257  (0.6%)

result           : PASS (converged to ground truth)
```

Perf

```
Performance counter stats for 'bin/openmp/icp_openmp_v0 500000':

     188162998880      cycles
     139817294240      instructions              #    0,74  insn per cycle
       1893803892      cache-references
        818198842      cache-misses              #   43,204 % of all cache refs
       2365431009      branch-misses
      25949613774      L1-dcache-loads
       5919388501      L1-dcache-load-misses     #   22,81% of all L1-dcache accesses
        772211262      LLC-loads
        316863579      LLC-load-misses           #   41,03% of all LL-cache accesses

      4,549863974 seconds time elapsed

     57,902078000 seconds user
      0,211327000 seconds sys
```


Flame Graph
Search
ic
nvompSchedulerAssignDy..
nvompSchedulerDispatchN..
__nvomp_GOMP_loop_dynam
n_search
icp_run._omp_fn.0
targetFuncHostTrampoline
targetFuncHostTrampoline_inline
targetFuncHostTrampoline_2
kernelEntryPoint
kernelEntryPoint
threadPoolEntryPoint
mainThreadPoolEntryPoint
start_thread
__GI___clone
icp_openmp_v0
b..
bu..
buil..
buil..
buil..
buil..
kd_b..
icp_run
main
__libc_star..
_start
n..
ic..
ta..
ta..
ta..
ke..
ke..
hx..
la..
la..
hx..
la..
ru..
nv..

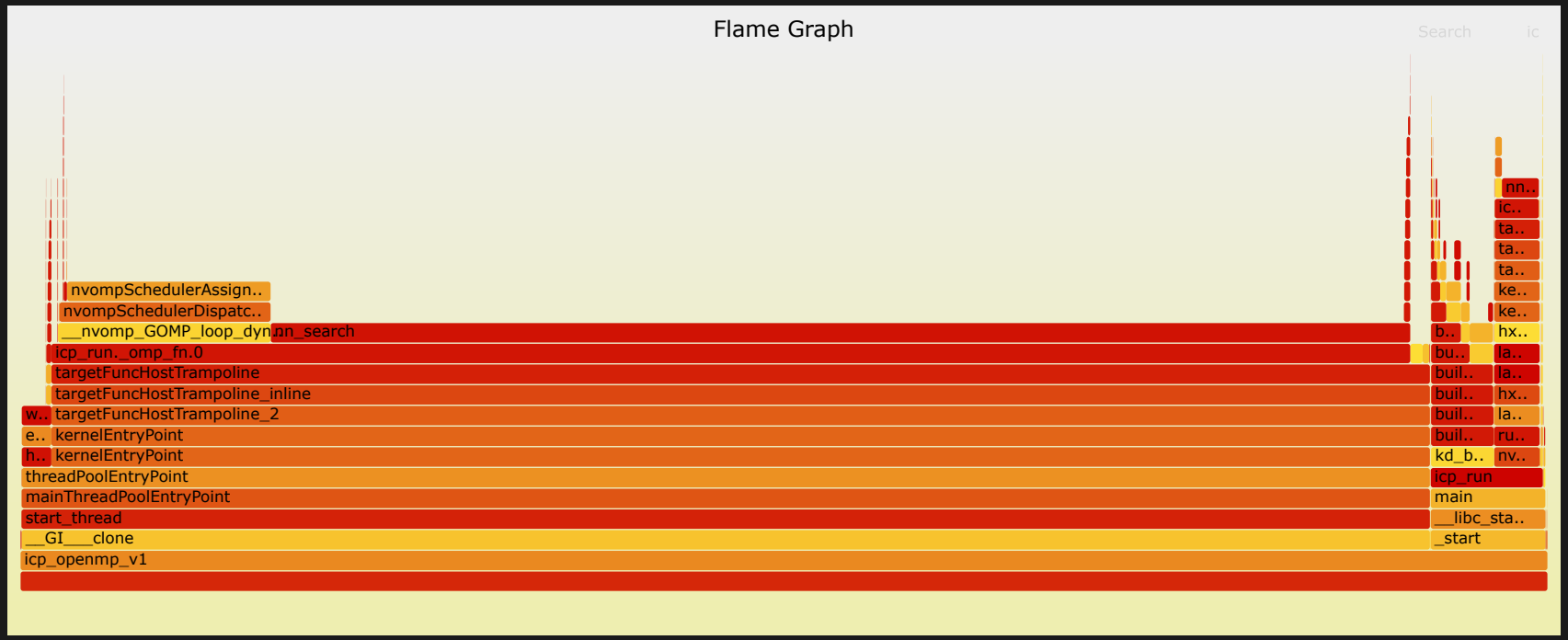
Parallelized the rest of ICP.

Output

```
==== Serial ICP (point-to-point, Kabsch/SVD) ====
backend          : KD-tree
target points    : 500000
source points    : 446322  (keep 85%, +5% outliers, noise 0.01 m)
seed             : 12345

ground-truth T   : [-0.4224  0.3500 -0.2812]
recovered    T   : [-0.4227  0.3470 -0.2781]

iterations       : 34
final RMSE       : 0.103125 m  over 440021 pairs
rotation error   : 0.0916 deg
translation err  : 0.0044 m

---- timing (s) ----
scene gen        :   0.0238
icp total        :   4.0481
  setup build    :   2.2252  (55.0%)
  NN search      :   1.7627  (43.5%)
  reduce+solve   :   0.0240  (0.6%)
  transform      :   0.0350  (0.9%)

result           : PASS (converged to ground truth)
```

Perf

```
Performance counter stats for 'bin/openmp/icp_openmp_v1 500000':

     189176003663      cycles
     140764760068      instructions              #    0,74  insn per cycle
       1899096068      cache-references
        788796131      cache-misses              #   41,535 % of all cache refs
       2379922077      branch-misses
      26203636320      L1-dcache-loads
       5910396282      L1-dcache-load-misses     #   22,56% of all L1-dcache accesses
        775277496      LLC-loads
        293660230      LLC-load-misses           #   37,88% of all LL-cache accesses

      4,178244755 seconds time elapsed

     58,464490000 seconds user
      0,180312000 seconds sys
```


Flame Graph
Search
ic
nn..
ic..
ta..
ta..
ta..
nvompSchedulerAssign..
ke..
nvompSchedulerDispatc..
ke..
__nvomp_GOMP_loop_dyn
nn_search
b..
hx..
icp_run._omp_fn.0
bu..
la..
targetFuncHostTrampoline
buil..
la..
targetFuncHostTrampoline_inline
buil..
hx..
w..
targetFuncHostTrampoline_2
buil..
la..
e..
kernelEntryPoint
buil..
ru..
h..
kernelEntryPoint
kd_b..
nv..
threadPoolEntryPoint
icp_run
mainThreadPoolEntryPoint
main
start_thread
__libc_sta..
__GI___clone
_start
icp_openmp_v1

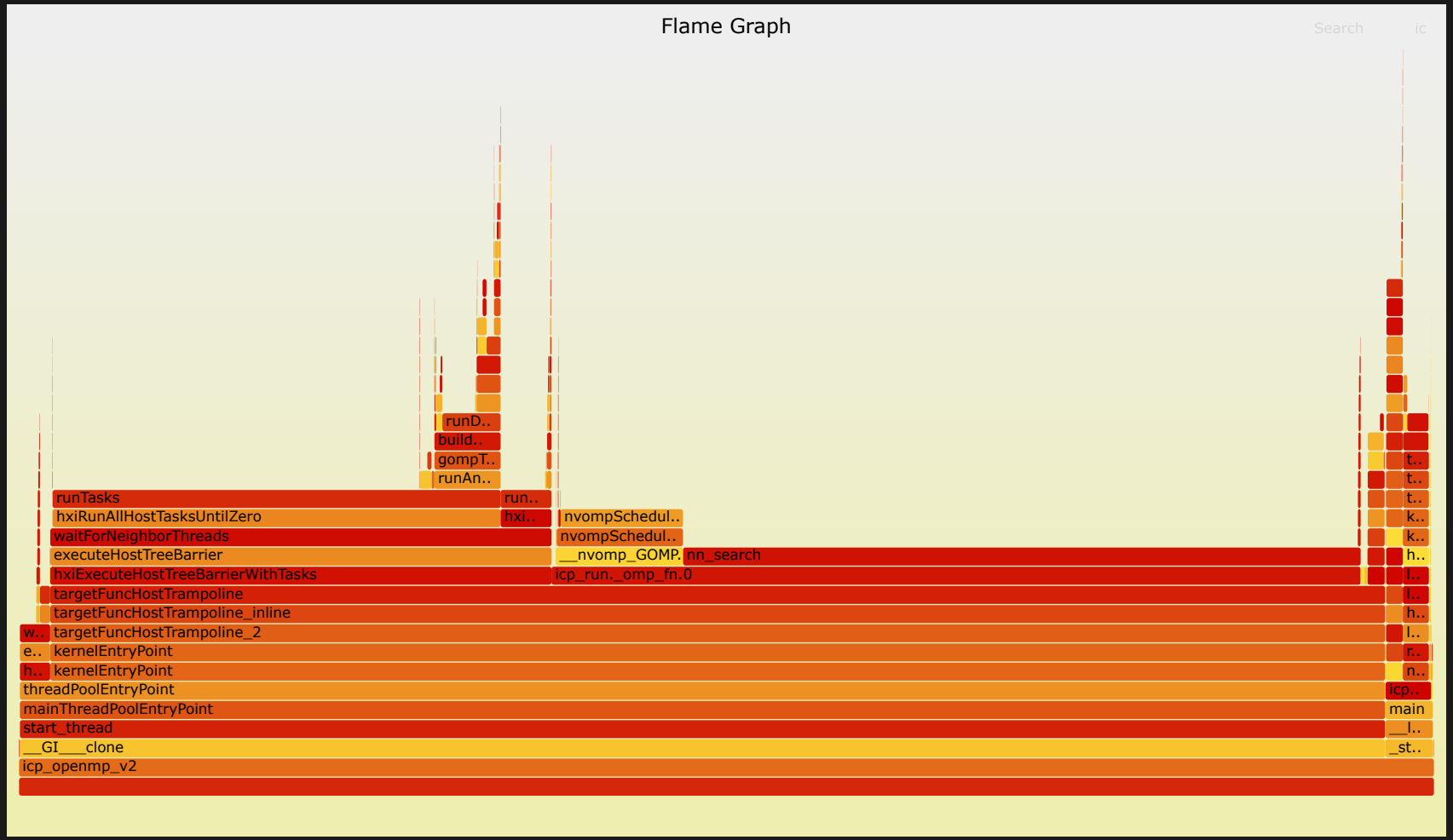
Parallelized the tree build with omp tasks.

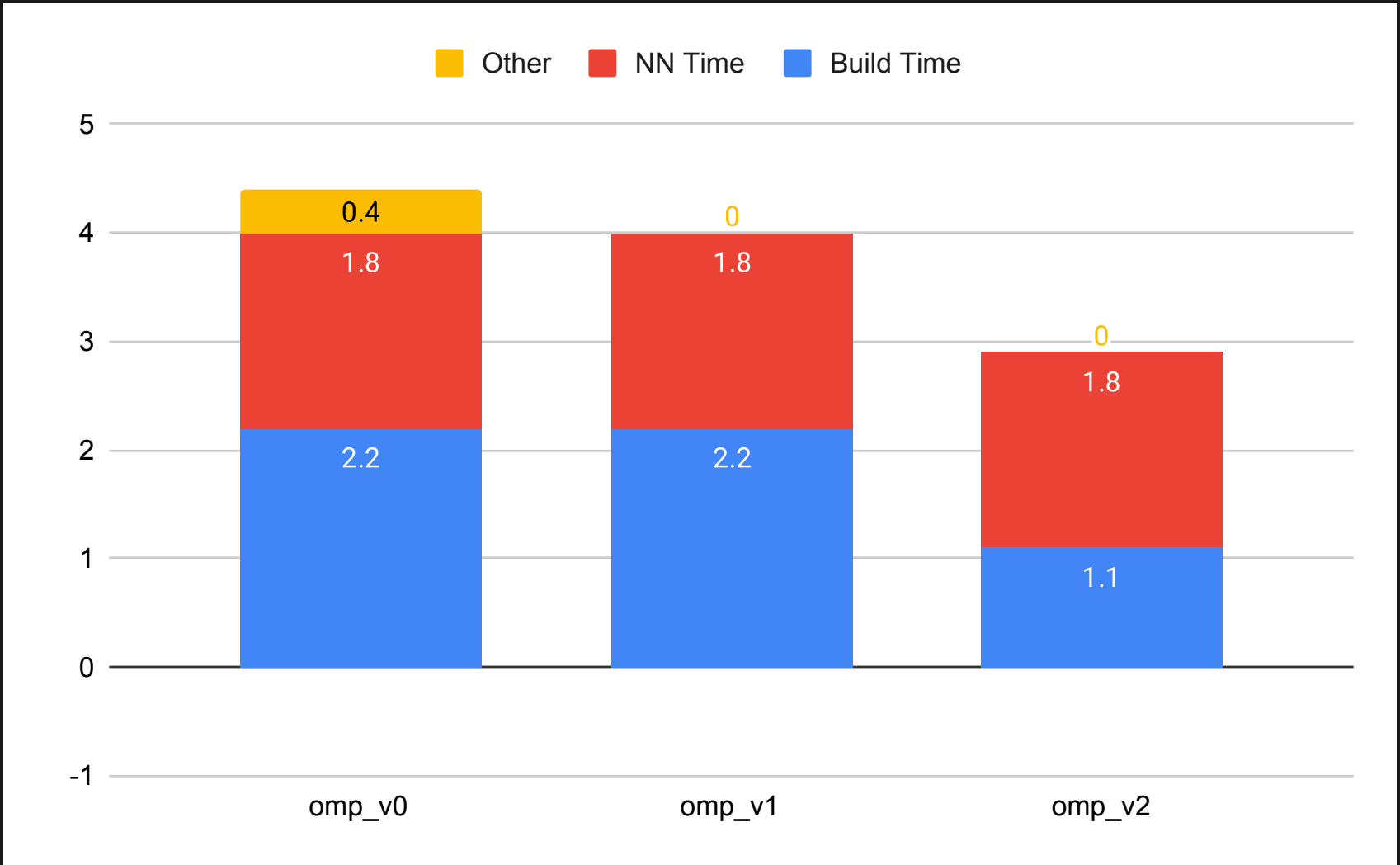
Output

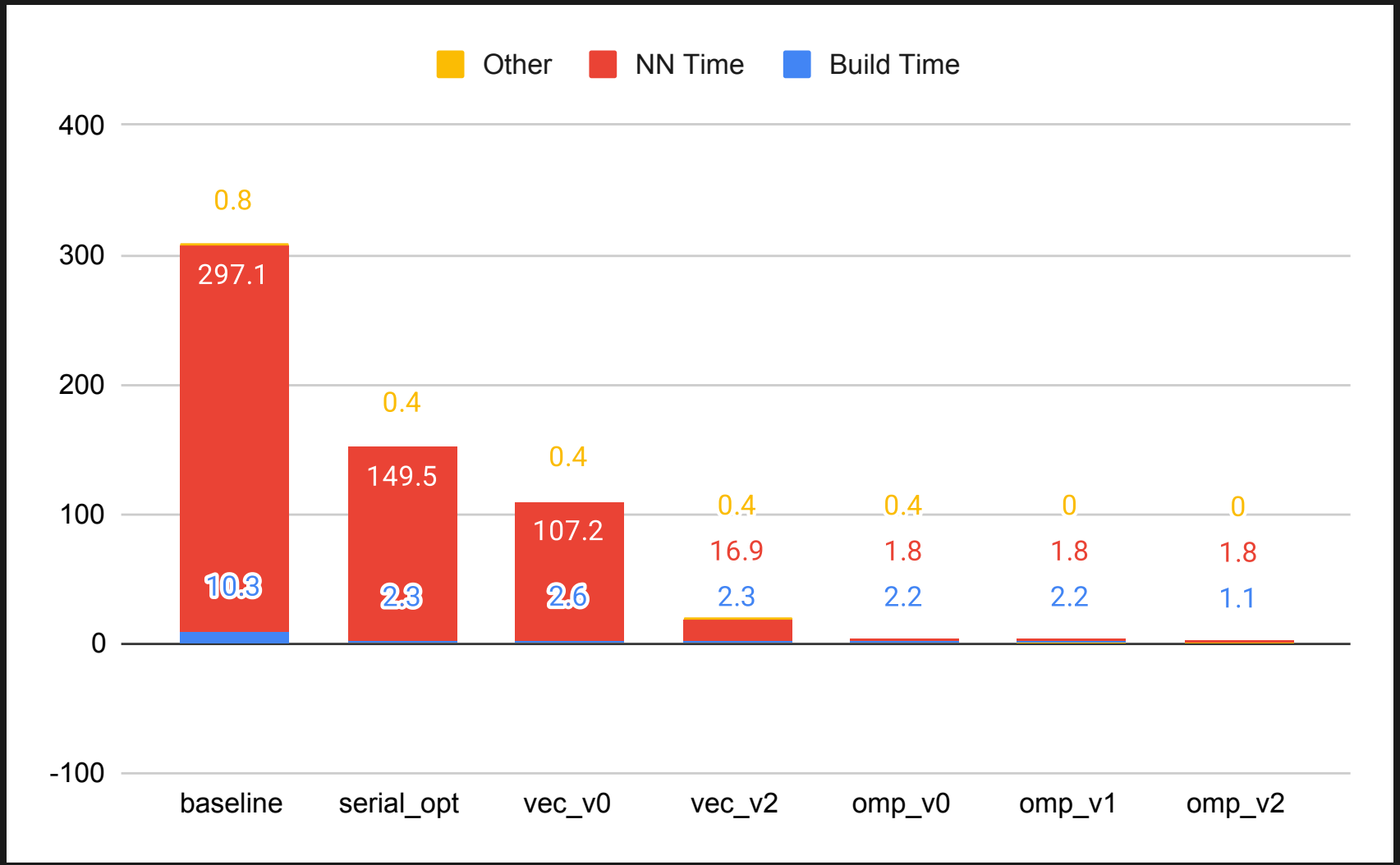
```
==== Serial ICP (point-to-point, Kabsch/SVD) ====
backend          : KD-tree
target points    : 500000
source points    : 446322  (keep 85%, +5% outliers, noise 0.01 m)
seed             : 12345

ground-truth T   : [-0.4224  0.3500 -0.2812]
recovered    T   : [-0.4227  0.3470 -0.2781]

iterations       : 34
final RMSE       : 0.103125 m  over 440021 pairs
rotation error   : 0.0916 deg
translation err  : 0.0044 m

---- timing (s) ----
scene gen        :   0.0238
icp total        :   2.9247
  setup build    :   1.0642  (36.4%)
  NN search      :   1.8102  (61.9%)
  reduce+solve   :   0.0178  (0.6%)
  transform      :   0.0311  (1.1%)

result           : PASS (converged to ground truth)
```

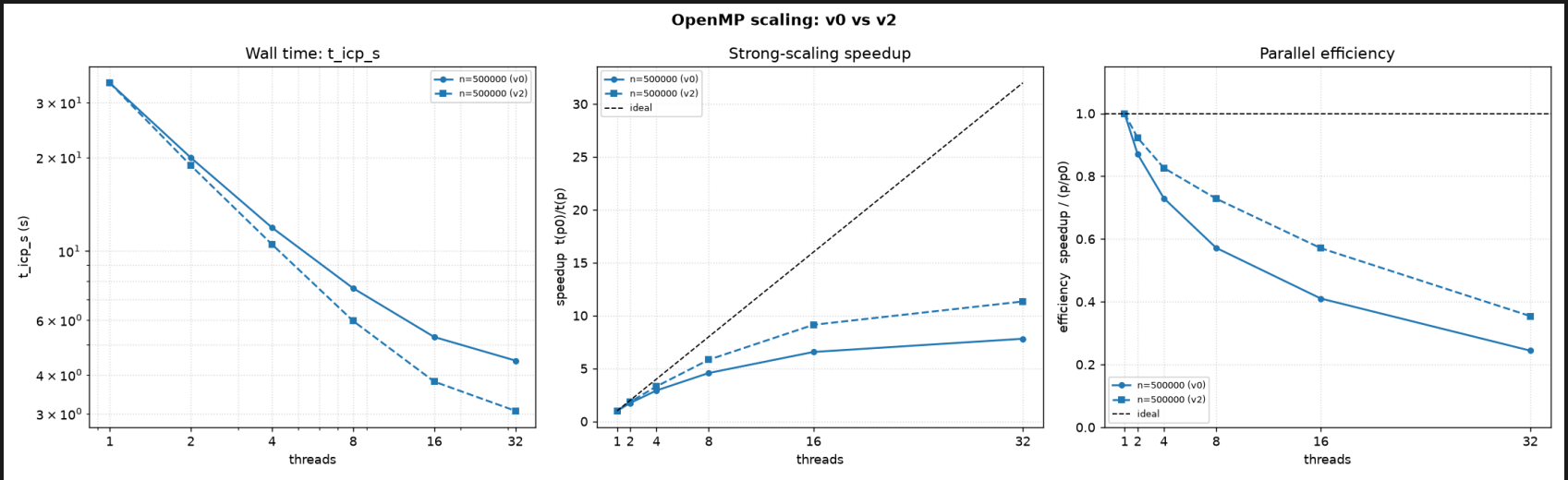
Perf

```
Performance counter stats for 'bin/openmp/icp_openmp_v2 500000':

     284678255166      cycles
     151771682975      instructions              #    0,53  insn per cycle
       4195120606      cache-references
       2020359995      cache-misses              #   48,160 % of all cache refs
       2375433039      branch-misses
      28782050163      L1-dcache-loads
       7238418857      L1-dcache-load-misses     #   25,15% of all L1-dcache accesses
       1687785430      LLC-loads
        887445693      LLC-load-misses           #   52,58% of all LL-cache accesses

      3,075129535 seconds time elapsed

     88,702054000 seconds user
      0,229714000 seconds sys
```


Flame Graph
Search
ic
runD..
build..
gompT..
runAn..
runTasks
run..
hxiRunAllHostTasksUntilZero
hxi..
nvompSchedul..
waitForNeighborThreads
nvompSchedul..
executeHostTreeBarrier
__nvomp_GOMP.
nn_search
hxiExecuteHostTreeBarrierWithTasks
icp_run._omp_fn.0
targetFuncHostTrampoline
targetFuncHostTrampoline_inline
w..
targetFuncHostTrampoline_2
e..
kernelEntryPoint
h..
kernelEntryPoint
threadPoolEntryPoint
mainThreadPoolEntryPoint
start_thread
__GI___clone
icp_openmp_v2
t..
t..
t..
k..
k..
h..
l..
l..
h..
l..
r..
n..
icp..
main
__l..
_st..


Other
NN Time
Build Time
5
4
3
2
1
0
-1
0.4
1.8
2.2
0
1.8
2.2
0
1.8
1.1
omp_v0
omp_v1
omp_v2


Other
NN Time
Build Time
400
300
200
100
0
-100
0.8
297.1
10.3
0.4
149.5
2.3
0.4
107.2
2.6
0.4
16.9
2.3
0.4
1.8
2.2
0
1.8
2.2
0
1.8
1.1
baseline
serial_opt
vec_v0
vec_v2
omp_v0
omp_v1
omp_v2


OpenMP scaling: v0 vs v2
Wall time: t_icp_s
t_icp_s (s)
threads
n=500000 (v0)
n=500000 (v2)
Strong-scaling speedup
speedup t(p0)/t(p)
threads
n=500000 (v0)
n=500000 (v2)
ideal
Parallel efficiency
efficiency speedup / (p/p0)
threads
n=500000 (v0)
n=500000 (v2)
ideal

5. GPU

Finally, the availability of GPUs allows for an even more massive parallelizatioin, so the code has been ported to CUDA and optimized for the GPU architecture at hand.

Output

```
==== Serial ICP (point-to-point, Kabsch/SVD) ====
backend          : KD-tree
target points    : 500000
source points    : 446322  (keep 85%, +5% outliers, noise 0.01 m)
seed             : 12345

ground-truth T   : [-0.4224  0.3500 -0.2812]
recovered    T   : [-0.4227  0.3470 -0.2781]

iterations       : 34
final RMSE       : 0.103125 m  over 440021 pairs
rotation error   : 0.0916 deg
translation err  : 0.0044 m

---- timing (s) ----
scene gen        :   0.0248
icp total        :   4.0616
  setup build    :   2.5321  (62.3%)
  CUDA ctx init  :   0.2918  (7.2%)
  H2D upload     :   0.0026  (0.1%)
  NN search      :   0.8124  (20.0%)
  reduce+solve   :   0.3839  (9.5%)
  transform      :   0.0377  (0.9%)

result           : PASS (converged to ground truth)
```

Nsys

[4/7] Executing 'cuda_api_sum' stats report

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
91,0	798941153	171	4672170,0	174902,0	122528	31271080	9067953,0	cudaMemcpy
8,0	70740150	7	10105735,0	61009,0	53852	70275643	26532470,0	cudaFree
0,0	942874	34	27731,0	22447,0	13226	194104	29658,0	cudaLaunchKernel
0,0	507263	6	84543,0	60455,0	57969	202808	58008,0	cudaMalloc
0,0	5780	1	5780,0	5780,0	5780	5780	0,0	cuModuleGetLoadingMode

[5/7] Executing 'cuda_gpu_kern_sum' stats report

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
100,0	764353009	34	22480970,0	21681339,0	21377565	31033131	1926800,0	kd_nearest_kernel

[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
68,0	10803726	103	104890,0	86240,0	76896	1893757	178165,0	[CUDA memcpy Host-to-Device]
31,0	4967386	68	73049,0	72479,0	70880	76288	1815,0	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
200,187	103	1,944	1,785	1,785	18,088	1,606	[CUDA memcpy Host-to-Device]
121,400	68	1,785	1,785	1,785	1,785	0,000	[CUDA memcpy Device-to-Host]

Ncu

```
Section: GPU Speed Of Light Throughput
----------------------- ------------- ------------
Metric Name               Metric Unit Metric Value
----------------------- ------------- ------------
DRAM Frequency          cycle/nsecond         1,59
SM Frequency            cycle/nsecond         1,24
Elapsed Cycles                  cycle     29086687
Memory Throughput                   %        17,23
DRAM Throughput                     %         3,03
Duration                       msecond        23,36
L1/TEX Cache Throughput             %        37,31
L2 Cache Throughput                 %        17,23
SM Active Cycles                cycle  12804207,59
Compute (SM) Throughput             %        11,72
----------------------- ------------- ------------

Section: Warp State Statistics
--------------------------------------- ----------- ------------
Metric Name                             Metric Unit Metric Value
--------------------------------------- ----------- ------------
Warp Cycles Per Issued Instruction             cycle        23,05
Warp Cycles Per Executed Instruction           cycle        23,08
Avg. Active Threads Per Warp                                 9,39
Avg. Not Predicated Off Threads Per Warp                     8,38
--------------------------------------- ----------- ------------

Section: Occupancy
------------------------------- ----------- ------------
Metric Name                     Metric Unit Metric Value
------------------------------- ----------- ------------
Block Limit SM                        block           32
Block Limit Registers                 block            8
Block Limit Shared Mem                block           32
Block Limit Warps                     block            8
Theoretical Active Warps per SM        warp           64
Theoretical Occupancy                     %          100
Achieved Occupancy                        %        38,35
Achieved Active Warps Per SM           warp        24,54
------------------------------- ----------- ------------
```

v1

Since nn search now is really fast and comparable with the other steps of ICP, the next step is to parallelize the rest of the algorithm. The tree build is left unchanged because is a one-time cost and not in the main focus for this work.

Also, to reduce `CudaMemcpy` overhead, ...

Output

```
==== Serial ICP (point-to-point, Kabsch/SVD) ====
backend          : KD-tree
target points    : 500000
source points    : 446322  (keep 85%, +5% outliers, noise 0.01 m)
seed             : 12345

ground-truth T   : [-0.4224  0.3500 -0.2812]
recovered    T   : [-0.4227  0.3470 -0.2781]

iterations       : 34
final RMSE       : 0.103125 m  over 440021 pairs
rotation error   : 0.0916 deg
translation err  : 0.0044 m

---- timing (s) ----
scene gen        :   0.0244
icp total        :   3.6088
  setup build    :   2.5719  (71.3%)
  CUDA ctx init  :   0.2664  (7.4%)
  H2D upload     :   0.0041  (0.1%)
  NN search      :   0.7587  (21.0%)
  reduce+solve   :   0.0055  (0.2%)
  transform      :   0.0009  (0.0%)

result           : PASS (converged to ground truth)
```

Nsys

[4/7] Executing 'cuda_api_sum' stats report

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
87,0	732550964	68	10772808,0	12248,0	9685	26946294	11196111,0	cudaDeviceSynchronize
8,0	69383122	18	3854617,0	54037,0	1874	68722893	16189022,0	cudaFree
3,0	26974124	204	132226,0	2580,0	2240	26132743	1829400,0	cudaLaunchKernel
1,0	8152201	245	33274,0	10441,0	3028	1759322	117131,0	cudaMemcpy
0,0	942894	17	55464,0	59263,0	1972	194143	56238,0	cudaMalloc
0,0	314527	170	1850,0	1521,0	1331	10217	1054,0	cudaMemset
0,0	4506	1	4506,0	4506,0	4506	4506	0,0	cuModuleGetLoadingMode

[5/7] Executing 'cuda_gpu_kern_sum' stats report

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
99,0	757934446	34	22292189,0	21764606,0	21304415	26936055	1345665,0	kd_nearest_kernel
0,0	1354300	34	39832,0	39456,0	37728	43968	1649,0	compute_cc
0,0	1082431	34	31836,0	31744,0	29984	33568	906,0	compute_centroids
0,0	228609	34	6723,0	6672,0	6464	7424	235,0	compute_match
0,0	202401	34	5953,0	5856,0	5664	6688	290,0	pc_transform_kernel
0,0	71422	34	2100,0	2080,0	1824	2528	185,0	average

[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
83,0	2492921	75	33238,0	992,0	959	1673949	194928,0	[CUDA memcpy Host-to-Device]
9,0	269984	170	1588,0	1536,0	1503	3008	150,0	[CUDA memcpy Device-to-Host]
7,0	236929	170	1393,0	1472,0	992	1888	254,0	[CUDA memset]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
29,447	75	0,393	0,000	0,000	18,088	2,135	[CUDA memcpy Host-to-Device]
0,004	170	0,000	0,000	0,000	0,000	0,000	[CUDA memcpy Device-to-Host]
0,004	170	0,000	0,000	0,000	0,000	0,000	[CUDA memset]

Ncu

```
Section: GPU Speed Of Light Throughput
----------------------- ------------- ------------
Metric Name               Metric Unit Metric Value
----------------------- ------------- ------------
DRAM Frequency          cycle/nsecond         1,59
SM Frequency            cycle/nsecond         1,24
Elapsed Cycles                  cycle     29896605
Memory Throughput                   %        16,68
DRAM Throughput                     %         3,00
Duration                       msecond        24,01
L1/TEX Cache Throughput             %        37,63
L2 Cache Throughput                 %        16,68
SM Active Cycles                cycle  12706374,46
Compute (SM) Throughput             %        11,40
----------------------- ------------- ------------

Section: Warp State Statistics
--------------------------------------- ----------- ------------
Metric Name                             Metric Unit Metric Value
--------------------------------------- ----------- ------------
Warp Cycles Per Issued Instruction            cycle        23,04
Warp Cycles Per Executed Instruction          cycle        23,07
Avg. Active Threads Per Warp                                9,39
Avg. Not Predicated Off Threads Per Warp                    8,38
--------------------------------------- ----------- ------------

Section: Occupancy
------------------------------- ----------- ------------
Metric Name                     Metric Unit Metric Value
------------------------------- ----------- ------------
Block Limit SM                        block           32
Block Limit Registers                 block            8
Block Limit Shared Mem                block           32
Block Limit Warps                     block            8
Theoretical Active Warps per SM        warp           64
Theoretical Occupancy                     %          100
Achieved Occupancy                        %        38,63
Achieved Active Warps Per SM           warp        24,72
------------------------------- ----------- ------------
```

As can be seen, only 9/32 warps are active at a time (most are masked due to divergence). Cache locality is the main problem, since warps are most of the time idle waiting for global memory loads.

The solution: memory optimization.

- KDTree with lower bytes;
- Smaller stack;
- Usage of `__restrict__`

KD-tree data structure for CUDA

```
1  typedef struct {
2      int count;
3      int axis;
4      float split;
5      int left, right;
6      int leaf_start;
7  } KDNodeGPU;
8
9  typedef struct {
10     const PointCloud *pts;
11     int *perm;
12     KDNodeGPU *nodes;
13     float *leaf_x, *leaf_y, *leaf_z;
14     int *leaf_idx;
15     int n;
16     int n_nodes;
17     int root;
18 } KDTreeGPU;
```

Output

```
==== Serial ICP (point-to-point, Kabsch/SVD) ====
backend          : KD-tree
target points    : 500000
source points    : 446322  (keep 85%, +5% outliers, noise 0.01 m)
seed             : 12345

ground-truth T   : [-0.4224  0.3500 -0.2812]
recovered    T   : [-0.4227  0.3470 -0.2781]

iterations       : 34
final RMSE       : 0.103125 m  over 440021 pairs
rotation error   : 0.0916 deg
translation err  : 0.0044 m

---- timing (s) ----
scene gen        :   0.0244
icp total        :   3.3846
  setup build    :   2.4748  (73.1%)
  CUDA ctx init  :   0.2709  (8.0%)
  H2D upload     :   0.0034  (0.1%)
  NN search      :   0.6280  (18.6%)
  reduce+solve   :   0.0057  (0.2%)
  transform      :   0.0009  (0.0%)

result           : PASS (converged to ground truth)
```

Nsys

[4/7] Executing 'cuda_api_sum' stats report

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
85,0	607779044	68	8937927,0	12079,0	9521	21417176	9295210,0	cudaDeviceSynchronize
10,0	72884587	22	3312935,0	55006,0	1995	72005464	15342723,0	cudaFree
3,0	21071549	204	103291,0	2628,0	2234	20225919	1415850,0	cudaLaunchKernel
1,0	7377938	249	29630,0	10571,0	3256	251768	43520,0	cudaMemcpy
0,0	1189583	21	56646,0	58238,0	2044	203093	51263,0	cudaMalloc
0,0	300419	170	1767,0	1492,0	1336	9527	872,0	cudaMemset
0,0	4404	1	4404,0	4404,0	4404	4404	0,0	cuModuleGetLoadingMode

[5/7] Executing 'cuda_gpu_kern_sum' stats report

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
99,0	623399203	34	18335270,0	17909556,0	17479556	21351519	1037767,0	kd_nearest_kernel
0,0	1375871	34	40466,0	39920,0	38305	44256	1847,0	compute_cc
0,0	1149368	34	33804,0	33520,0	32224	36256	1069,0	compute_centroids
0,0	228321	34	6715,0	6656,0	6367	7392	235,0	compute_match
0,0	205311	34	6038,0	5888,0	5664	6912	393,0	pc_transform_kernel
0,0	67139	34	1974,0	1921,0	1792	2304	143,0	average

[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
71,0	1286399	79	16283,0	992,0	959	140608	39802,0	[CUDA memcpy Host-to-Device]
15,0	269663	170	1586,0	1536,0	1503	2336	113,0	[CUDA memcpy Device-to-Host]
13,0	235005	170	1382,0	1472,0	992	1920	268,0	[CUDA memset]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
19,752	79	0,250	0,000	0,000	2,000	0,648	[CUDA memcpy Host-to-Device]
0,004	170	0,000	0,000	0,000	0,000	0,000	[CUDA memcpy Device-to-Host]
0,004	170	0,000	0,000	0,000	0,000	0,000	[CUDA memset]

Ncu

```
Section: GPU Speed Of Light Throughput
----------------------- ------------- ------------
Metric Name               Metric Unit Metric Value
----------------------- ------------- ------------
DRAM Frequency          cycle/nsecond         1,59
SM Frequency            cycle/nsecond         1,24
Elapsed Cycles                  cycle     23500378
Memory Throughput                   %        18,38
DRAM Throughput                     %         0,61
Duration                       msecond        18,88
L1/TEX Cache Throughput             %        46,61
L2 Cache Throughput                 %         8,03
SM Active Cycles                cycle   9268088,19
Compute (SM) Throughput             %        14,47
----------------------- ------------- ------------

Section: Warp State Statistics
--------------------------------------- ----------- ------------
Metric Name                             Metric Unit Metric Value
--------------------------------------- ----------- ------------
Warp Cycles Per Issued Instruction            cycle        16,53
Warp Cycles Per Executed Instruction          cycle        16,54
Avg. Active Threads Per Warp                               10,67
Avg. Not Predicated Off Threads Per Warp                    9,34
--------------------------------------- ----------- ------------

Section: Occupancy
------------------------------- ----------- ------------
Metric Name                     Metric Unit Metric Value
------------------------------- ----------- ------------
Block Limit SM                        block           32
Block Limit Registers                 block            8
Block Limit Shared Mem                block           32
Block Limit Warps                     block            8
Theoretical Active Warps per SM        warp           64
Theoretical Occupancy                     %          100
Achieved Occupancy                        %        35,71
Achieved Active Warps Per SM           warp        22,85
------------------------------- ----------- ------------
```

Other small optimizations could be done, but the main bottleneck remains the tree structure. Using a KD-tree is a useful way to speed up the search, but it is very difficult to parallelize in the GPU architecture.

But there is a catch: if one has knowledge of the distribution of the points, a **voxel grid** can be used as data structure.

v3

```
typedef struct {
    int n;           // target points
    int bits;        // number of bits per axis
    int G;           // cells per axis = 1<<bits = 1*2^bits
    long nv;         // total voxels = (long)G*G*G
    double origin[3];   // target bbox min
    double inv_cell[3]; // 1/cell_size per axis: v = floor((p-o)*inv)

    float *x;
    float *y;
    float *z;
    int *offsets;
    int *voxel_root;
    int *idx;
} VoxelGrid;
```

Output

```
==== CUDA v3 ICP (point-to-point, Kabsch/SVD) ====
backend          : voxel grid
target points    : 500000
source points    : 446322  (keep 85%, +5% outliers, noise 0.01 m)
seed             : 12345

ground-truth T   : [-0.4224  0.3500 -0.2812]
recovered    T   : [-0.4230  0.3465 -0.2781]

iterations       : 35
final RMSE       : 0.090484 m  over 438199 pairs
rotation error   : 0.0950 deg
translation err  : 0.0047 m

---- timing (s) ----
scene gen        :   0.0396
icp total        :   0.3823
  setup build    :   0.1020  (26.7%)
  CUDA ctx init  :   0.2665  (69.7%)
  H2D upload     :   0.0034  (0.9%)
  NN search      :   0.0026  (0.7%)
  reduce+solve   :   0.0060  (1.6%)
  transform      :   0.0010  (0.3%)

result           : PASS (converged to ground truth)
```

Nsys

[4/7] Executing 'cuda_api_sum' stats report

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
77,0	71619656	36	1989434,0	54815,0	2061	70084743	11673518,0	cudaFree
15,0	14077329	268	52527,0	10862,0	3098	1155126	140834,0	cudaMemcpy
2,0	2549785	70	36425,0	12017,0	8550	70080	25936,0	cudaDeviceSynchronize
2,0	1827551	35	52215,0	59390,0	1673	151700	36442,0	cudaMalloc
1,0	1582962	224	7066,0	2508,0	2204	589215	42287,0	cudaLaunchKernel
0,0	304277	176	1728,0	1465,0	1319	25926	1868,0	cudaMemset
0,0	10716	1	10716,0	10716,0	10716	10716	0,0	cudaStreamSynchronize
0,0	3728	1	3728,0	3728,0	3728	3728	0,0	cuModuleGetLoadingMode

[5/7] Executing 'cuda_gpu_kern_sum' stats report

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
36,0	1940639	35	55446,0	55488,0	51680	63072	1857,0	voxel_nn_kernel
28,0	1531836	35	43766,0	43584,0	42080	45279	879,0	compute_cc
21,0	1156732	35	33049,0	33119,0	30111	35424	1391,0	compute_centroids
4,0	230719	35	6592,0	6592,0	6432	6752	86,0	pc_transform_kernel
3,0	172161	35	4918,0	4928,0	4640	5312	151,0	compute_match
1,0	75456	35	2155,0	2144,0	2016	2336	113,0	average
1,0	53568	9	5952,0	5952,0	5760	6336	183,0	dilate_kernel

[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

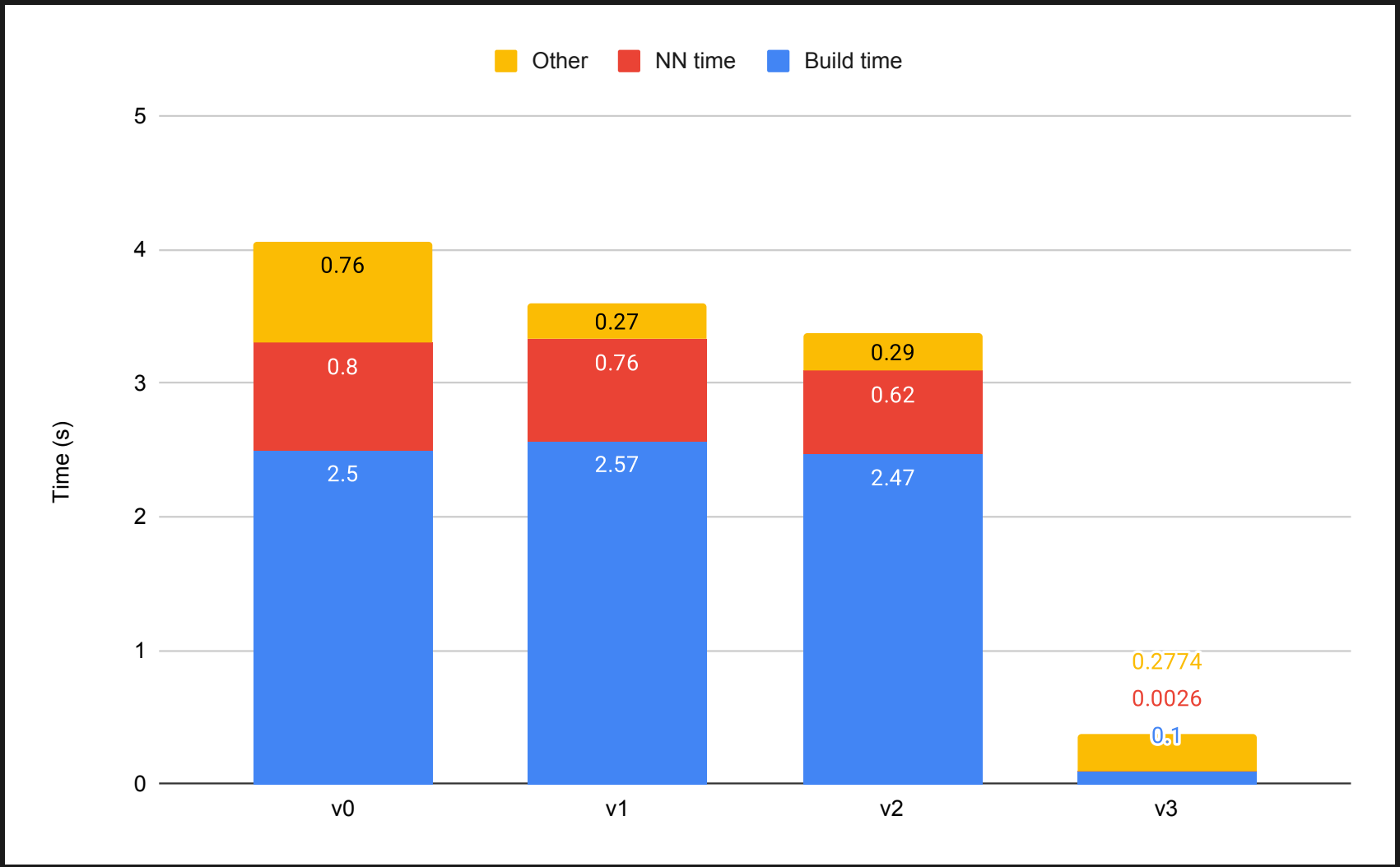
Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
64,0	1742626	86	20263,0	1121,0	1087	131168	43092,0	[CUDA memcpy Host-to-Device]
25,0	700255	181	3868,0	1664,0	1632	80544	12298,0	[CUDA memcpy Device-to-Host]
10,0	274531	176	1559,0	1600,0	1119	3104	258,0	[CUDA memset]
0,0	3552	1	3552,0	3552,0	3552	3552	0,0	[CUDA memcpy Device-to-Device]

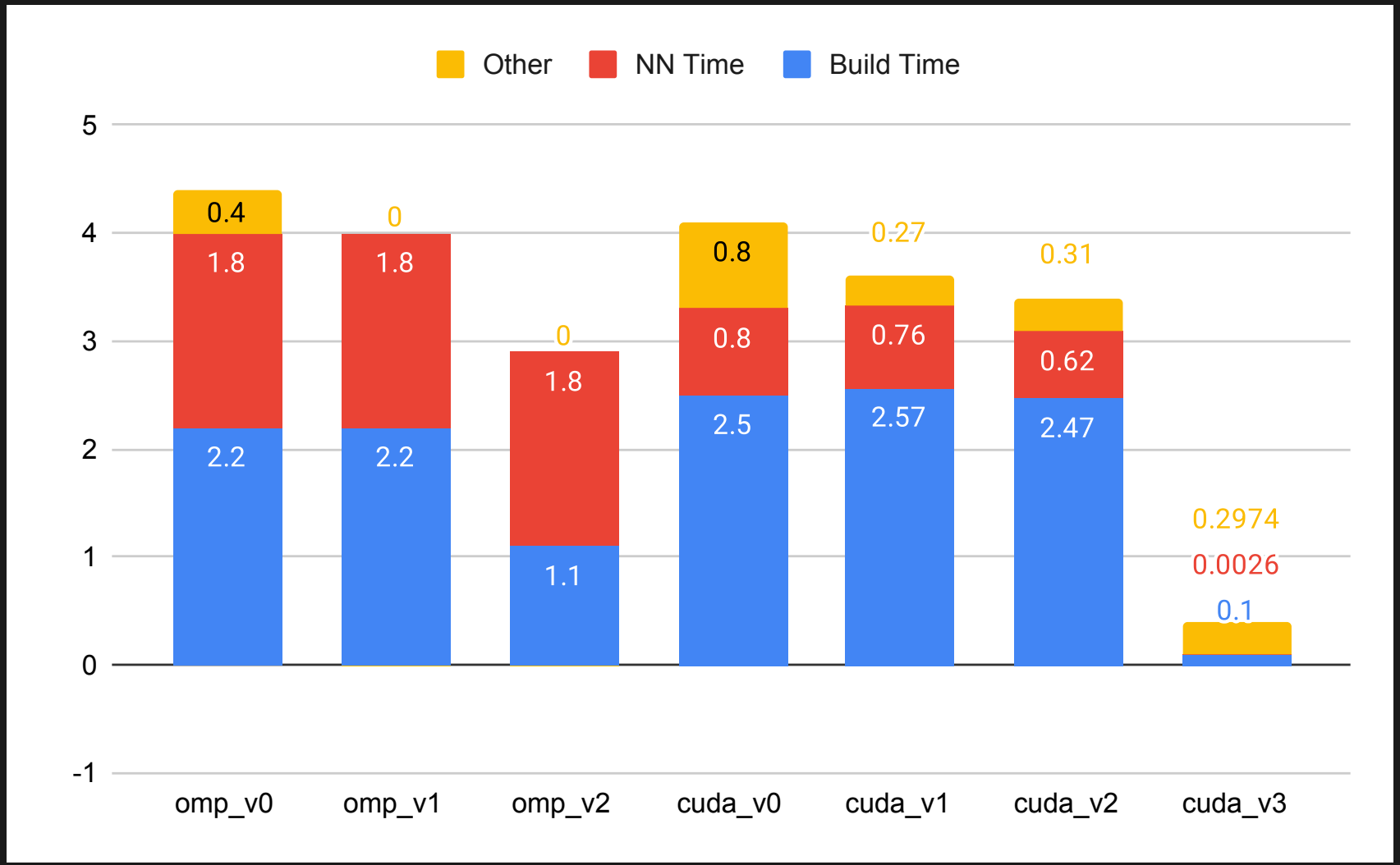
[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

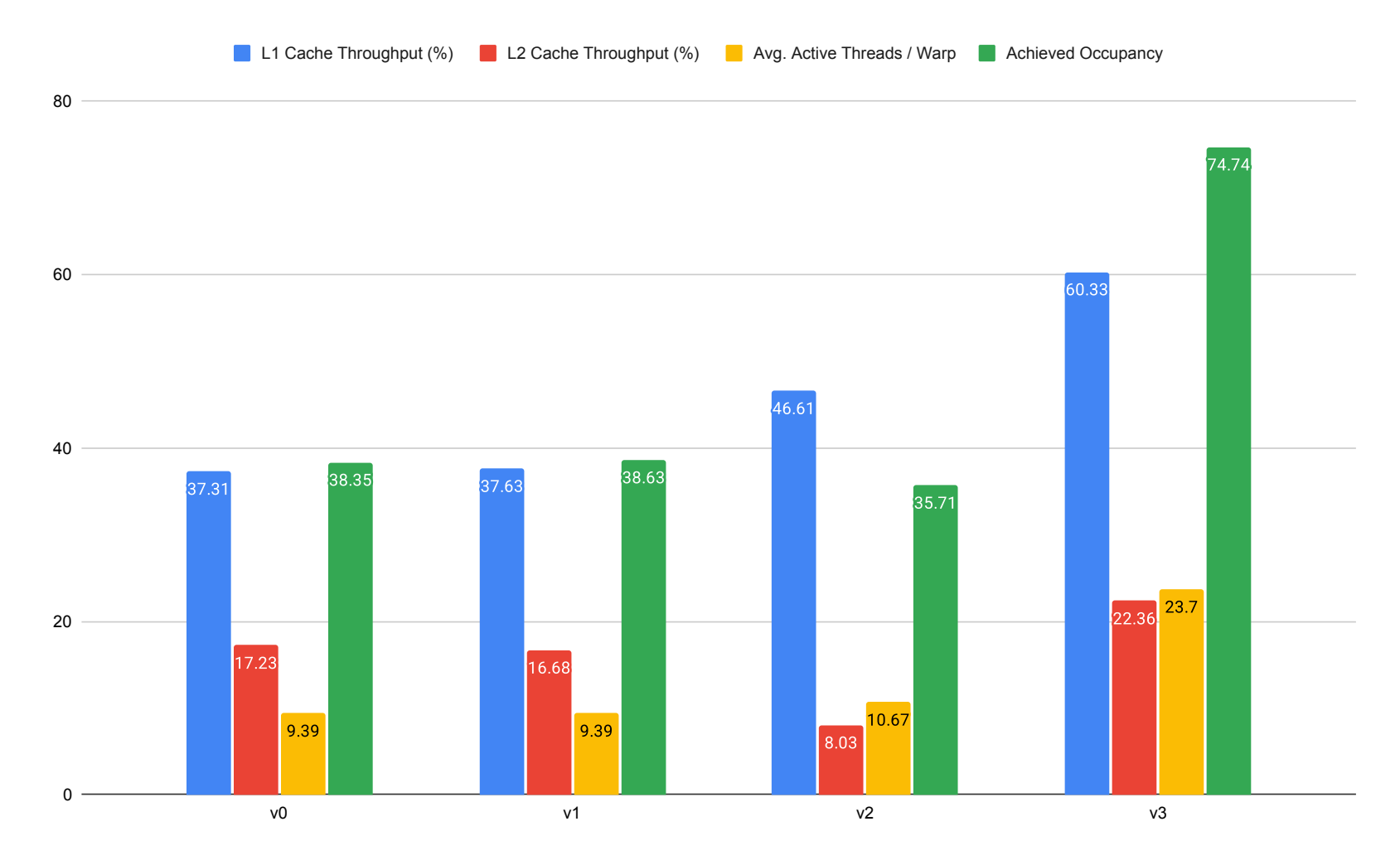
Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
27,456	86	0,319	0,000	0,000	2,000	0,711	[CUDA memcpy Host-to-Device]
10,102	181	0,056	0,000	0,000	2,000	0,313	[CUDA memcpy Device-to-Host]
1,053	176	0,006	0,000	0,000	1,049	0,079	[CUDA memset]
1,049	1	1,049	1,049	1,049	1,049	0,000	[CUDA memcpy Device-to-Device]

Ncu

```
Section: GPU Speed Of Light Throughput
----------------------- ------------- ------------
Metric Name               Metric Unit Metric Value
----------------------- ------------- ------------
DRAM Frequency          cycle/nsecond         1,59
SM Frequency            cycle/nsecond         1,23
Elapsed Cycles                  cycle        71013
Memory Throughput                   %        39,07
DRAM Throughput                     %        16,07
Duration                       usecond        57,47
L1/TEX Cache Throughput             %        60,33
L2 Cache Throughput                 %        22,36
SM Active Cycles                cycle     45826,15
Compute (SM) Throughput             %        26,88
----------------------- ------------- ------------

Section: Warp State Statistics
--------------------------------------- ----------- ------------
Metric Name                             Metric Unit Metric Value
--------------------------------------- ----------- ------------
Warp Cycles Per Issued Instruction             cycle        28,78
Warp Cycles Per Executed Instruction           cycle        28,87
Avg. Active Threads Per Warp                                23,70
Avg. Not Predicated Off Threads Per Warp                    21,16
--------------------------------------- ----------- ------------

Section: Occupancy
------------------------------- ----------- ------------
Metric Name                     Metric Unit Metric Value
------------------------------- ----------- ------------
Block Limit SM                        block           32
Block Limit Registers                 block            8
Block Limit Shared Mem                block           32
Block Limit Warps                     block            8
Theoretical Active Warps per SM        warp           64
Theoretical Occupancy                     %          100
Achieved Occupancy                        %        74,74
Achieved Active Warps Per SM           warp        47,84
------------------------------- ----------- ------------
```


Other
NN time
Build time
Time (s)
5
4
3
2
1
0
0.76
0.8
2.5
0.27
0.76
2.57
0.29
0.62
2.47
0.2774
0.0026
0.1
v0
v1
v2
v3


Other
NN Time
Build Time
5
4
3
2
1
0
-1
0.4
1.8
2.2
0
1.8
2.2
0
1.8
1.1
0.8
0.8
2.5
0.27
0.76
2.57
0.31
0.62
2.47
0.2974
0.0026
0.1
omp_v0
omp_v1
omp_v2
cuda_v0
cuda_v1
cuda_v2
cuda_v3


L1 Cache Throughput (%)
L2 Cache Throughput (%)
Avg. Active Threads / Warp
Achieved Occupancy
80
60
40
20
0
37.31
17.23
9.39
38.35
37.63
16.68
9.39
38.63
46.61
8.03
10.67
35.71
60.33
22.36
23.7
74.74
v0
v1
v2
v3

6. Conclusions

Conclusions

- Effective parallelization of the ICP algorithm is necessary for real-time applications, and it can be achieved in different ways;
- Parallelization of a KD-tree is possible, even though not as efficient as a voxel grid, but without prior knowledge of the point distribution, it is the only option;
- A parallel implementation with KD-tree always beats a parallel brute implementation.

7. References

References

Qiong Chang, Weimin Wang, and Jun Miyazaki. 2025. *Accelerating Nearest Neighbor Search in 3D Point Cloud Registration on GPUs*. ACM Trans. Arch. Code Optim. 22, 1, Article 43 (March 2025), 24 pages. https://doi.org/10.1145/3716875

Matthias Kretz. 2015. *Extending C++ for Explicit Data-Parallel Programming via SIMD Vector Types*. Dissertation, Goethe University Frankfurt.